

- SPC-PIL — Software Documentation
 - Table of Contents
 - 1. Project Overview
 - Key Capabilities (Planned Modules)
 - 2. Tech Stack
 - Core Framework & Language
 - Styling & UI
 - Forms & Validation
 - Data & State
 - Date & Table
 - Font
 - 3. Project Structure
 - 4. Application Flow
 - High-Level Application Flow
 - 5. Route Architecture
 - Public Routes
 - Protected Routes (Authenticated Group)
 - 6. Authentication Module
 - 6.1 Auth State Machine
 - 6.2 Auth States — Detailed
 - 6.3 OTPInput Component
 - 6.4 Resend OTP Cooldown Logic
 - 6.5 Auth Page Layout
 - 7. Layout System
 - 7.1 Root Layout (app/layout.tsx)
 - 7.2 Authenticated Layout (app/(authenticated)/layout.tsx)
 - 7.3 Sidebar Component (components/layout/Sidebar.tsx)
 - 7.4 Header Component (components/layout/Header.tsx)
 - 8. Dashboard Module
 - 8.1 Dashboard Page Composition
 - 8.2 SearchDocument Component
 - 8.3 Cards Component
 - 9. API Layer
 - 9.1 Architecture
 - 9.2 mainApiCaller Function
 - 9.3 Mock API Endpoints
 - 9.4 Real API Implementation (Commented)

- 9.5 Simulated API Delay
- 10. Component Library (UI)
 - Button Component (components/ui/button.tsx)
 - Card Component (components/ui/card.tsx)
 - Form Components
 - Utility Function (lib/Utils.ts)
- 11. Design System
 - 11.1 Color Palette
 - 11.2 Gradient System
 - 11.3 Typography
 - 11.4 Spacing & Radius
 - 11.5 Dark Mode
 - 11.6 Custom Scrollbar
- 12. Navigation Configuration
- 13. Mock Data & Dev Credentials
 - Test User Accounts
 - OTP (All Users)
 - Dashboard Mock Data
- 14. Environment & Configuration
 - TypeScript Config (tsconfig.json)
 - Path Alias
 - Shadcn Config (components.json)
 - Next.js Config (next.config.ts)
- 15. Scripts & Commands
 - Development Server
- Appendix: Data Flow Diagram

SPC-PIL — Software Documentation

DDReg Pharma | Regulatory Document Management Platform Version: 0.1.0 |
Framework: Next.js 16 | Language: TypeScript

Table of Contents

1. [Project Overview](#)

- 2. [Tech Stack](#)
- 3. [Project Structure](#)
- 4. [Application Flow](#)
- 5. [Route Architecture](#)
- 6. [Authentication Module](#)
- 7. [Layout System](#)
- 8. [Dashboard Module](#)
- 9. [API Layer](#)
- 10. [Component Library \(UI\)](#)
- 11. [Design System](#)
- 12. [Navigation Configuration](#)
- 13. [Mock Data & Dev Credentials](#)
- 14. [Environment & Configuration](#)
- 15. [Scripts & Commands](#)

1. Project Overview

SPC-PIL is a web-based **Regulatory Document Management Platform** developed for **DDReg Pharma**. The platform is designed to manage two critical types of pharmaceutical regulatory documents:

Term	Full Form	Description
SPC	Summary of Product Characteristics	A detailed technical document about a medicinal product for healthcare professionals
PIL	Patient Information Leaflet	A document written for patients describing how to use a medicine safely

Key Capabilities (Planned Modules)

-  Secure user authentication with 2-Factor Authentication (2FA)
-  AI-powered SPC / PIL document generation
-  Rich document editor for regulatory content
-  Workflow management for document approval processes
-  Document repository with version control
-  Template library for standard document formats

-  Regulatory intelligence for country-specific guidelines
-  Document comparison tools
-  Audit & version history tracking
-  Notification center for alerts & approvals

2. Tech Stack

Core Framework & Language

Technology	Version	Purpose
Next.js	16.1.6	Full-stack React framework (App Router)
React	19.2.3	UI rendering
TypeScript	^5	Type-safe JavaScript

Styling & UI

Technology	Version	Purpose
Tailwind CSS	^4	Utility-first CSS framework
Shadcn/UI	^3.8.4	Pre-built accessible component library
Radix UI	^1.4.3	Headless UI primitives
tw-animate-css	^1.4.0	Animation utilities for Tailwind
Framer Motion	^12.33.0	Smooth animations & transitions
Lucide React	^0.563.0	Icon library
class-variance-authority	^0.7.1	Component variant management
clsx + tailwind-merge	latest	Conditional className utilities

Forms & Validation

Technology	Version	Purpose
React Hook Form	^7.71.1	Performant form management
Zod	^4.3.6	Schema-based form validation
@hookform/resolvers	^5.2.2	Connects Zod with React Hook Form

Data & State

Technology	Version	Purpose
SWR	^2.4.0	Data fetching & caching
react-hot-toast	^2.6.0	Toast notifications

Date & Table

Technology	Version	Purpose
date-fns	^4.1.0	Date utility functions
react-day-picker	^9.13.2	Calendar/date picker component
@tanstack/react-table	^8.21.3	Powerful headless table library
vaul	^1.1.2	Drawer/sheet component

Font

- **Poppins** (Google Fonts) — Weights: 100 to 900

3. Project Structure

spc-pil/	
— app/	# Next.js App Router
— favicon.ico	
— globals.css	# Global styles, CSS variables, design tokens
— layout.tsx	# Root layout (Font, Toaster,

TooltipProvider)	
├── page.tsx	# Public root page → Renders Auth/Login page
├── (authenticated)/	# Route Group – Protected pages
│ ├── layout.tsx	# Authenticated layout (Sidebar + Header +
Content)	
│ ├── dashboard/	
│ ├── page.tsx	# Dashboard page
│ ├── account/	
│ ├── page.tsx	# User account page
│ ├── generate-SPC-PIL/	
│ ├── page.tsx	# SPC/PIL document generation
│ ├── document-editor/	
│ ├── page.tsx	# Rich document editor
│ ├── workflow-management/	
│ ├── page.tsx	# Workflow management
│ ├── document-repository/	
│ ├── page.tsx	# Document repository
│ ├── template-library/	
│ ├── page.tsx	# Template library
│ ├── regulatory-intelligence/	
│ ├── page.tsx	# Regulatory intelligence
│ ├── compare-documents/	
│ ├── page.tsx	# Document comparison
│ ├── audit-version-history/	
│ ├── page.tsx	# Audit & version history
│ ├── notification-center/	
│ ├── page.tsx	# Notification center
│ ├── settings/	
│ ├── page.tsx	# Settings
├── features/	# Feature-based modules (business logic)
│ ├── auth/	
│ ├── index.tsx	# AuthPage – main export (layout wrapper)
│ ├── components/	
│ ├── authForm.tsx	# AuthForm – full auth state machine
│ ├── OTPInput.tsx	# Custom 6-digit OTP input component
│ ├── types/	# (Reserved for auth TypeScript types)
│ ├── dashboard/	
│ ├── index.tsx	# DashboardPage – main export
│ ├── components/	
│ ├── Cards.tsx	# Stat cards component
│ ├── types/	# (Reserved for dashboard TypeScript types)
├── components/	# Shared reusable components
│ ├── layout/	
│ ├── Header.tsx	# Dynamic page header (title + search +
actions)	
│ ├── Sidebar.tsx	# Collapsible navigation sidebar
│ ├── common/	
│ ├── SearchDocument.tsx	# Document search with filters
│ ├── ui/	# Shadcn/Radix UI component library
│ ├── button.tsx	
│ ├── card.tsx	
│ ├── input.tsx	
│ ├── form.tsx	

```

├── select.tsx
├── tooltip.tsx
├── dialog.tsx
├── sheet.tsx
├── drawer.tsx
├── table.tsx
├── tabs.tsx
├── badge.tsx
├── avatar.tsx
├── calendar.tsx
├── checkbox.tsx
├── radio-group.tsx
├── switch.tsx
├── textarea.tsx
├── skeleton.tsx
├── scroll-area.tsx
├── separator.tsx
├── label.tsx
├── field.tsx
├── item.tsx
├── input-group.tsx
├── button-group.tsx
├── aspect-ratio.tsx
├── lib/
│   ├── NavData.ts
│   ├── utils.ts
│   └── apis/
│       ├── apicaller.ts
│       └── mockData.ts
├── public/
│   ├── auth/
│   ├── dashboard/cards/
│   ├── header/black/ & white/
│   ├── search-document/
│   └── sidebar/
│       ├── black/
│       ├── white/
│       ├── header/
│       └── footer/
├── components.json
├── next.config.ts
├── tailwind.config (via postcss)
├── tsconfig.json
├── eslint.config.mjs
├── package.json

```

4. Application Flow

High-Level Application Flow



▼	
Content (children)	Page
(Dashboard, Editor, etc.)	

5. Route Architecture

Public Routes

Route	Page File	Description
/	app/page.tsx	Login / Authentication page

Protected Routes (Authenticated Group)

All routes under `app/(authenticated)/` share a common layout that includes the **Sidebar** and **Header**.

Route	Page File	Nav Title	Status
/dashboard	dashboard/page.tsx	Dashboard	 Implemented
/generate-SPC-PIL	generate-SPC-PIL/page.tsx	Generate SPC / PIL	 Placeholder
/document-editor	document-editor/page.tsx	Document Editor	 Placeholder
/workflow-management	workflow-management/page.tsx	Workflow Management	 Placeholder
/document-repository	document-repository/page.tsx	Document Repository	 Placeholder
/template-library	template-library/page.tsx	Template Library	 Placeholder

Route	Page File	Nav Title	Status
/regulatory-intelligence	regulatory-intelligence/page.tsx	Regulatory Intelligence	 Placeholder
/compare-documents	compare-documents/page.tsx	Compare Documents	 Placeholder
/audit-version-history	audit-version-history/page.tsx	Audit & Version History	 Placeholder
/notification-center	notification-center/page.tsx	Notification Center	 Placeholder
/settings	settings/page.tsx	Settings	 Placeholder
/account	account/page.tsx	Account	 Placeholder (Hidden in nav)

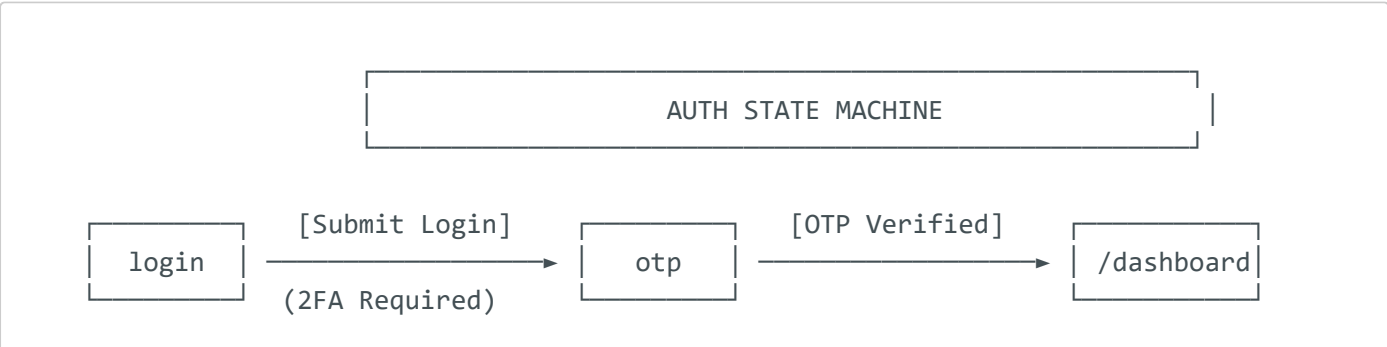
Note: `(authenticated)` is a Next.js **Route Group** — the parentheses mean it does NOT affect the URL path. It only groups routes to apply a shared layout.

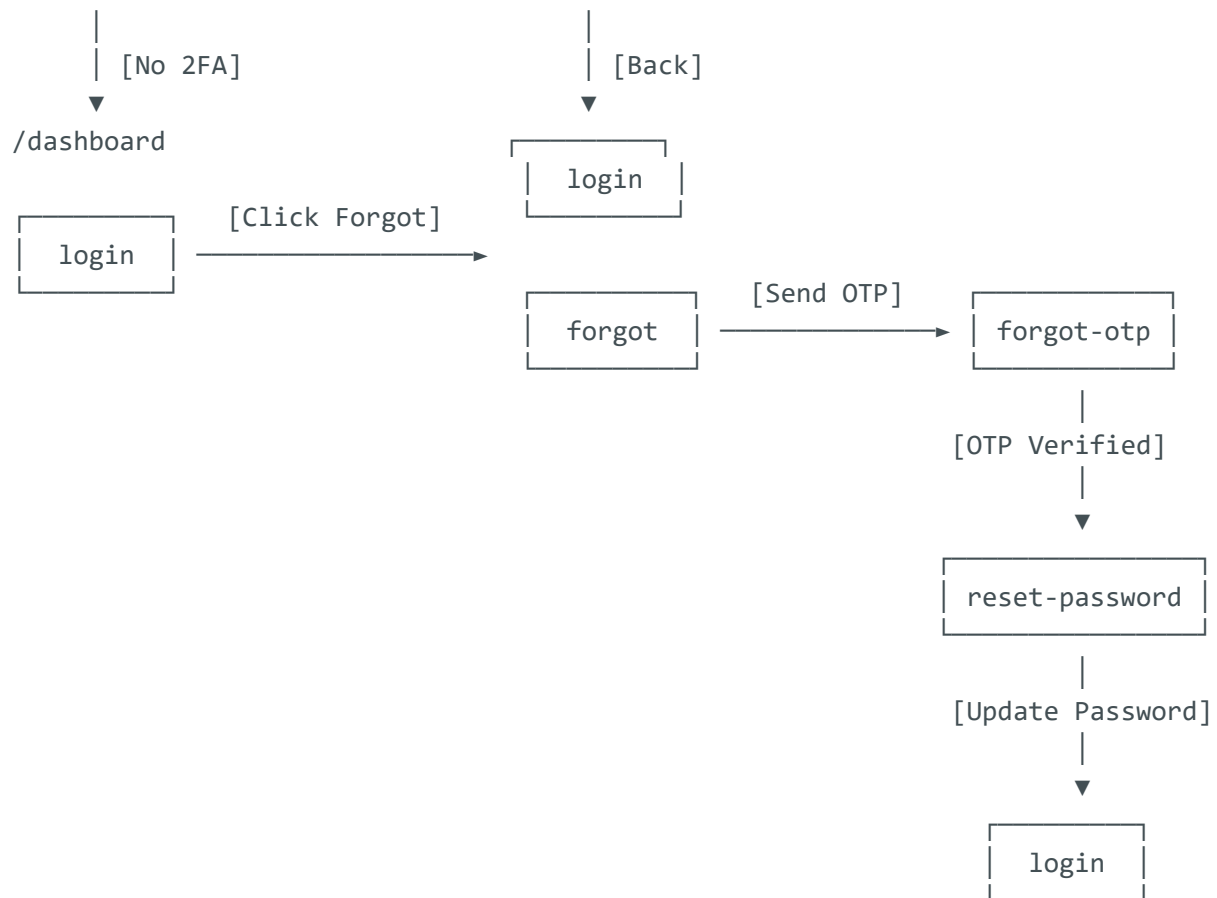
6. Authentication Module

Location: `features/auth/`

6.1 Auth State Machine

The `AuthForm` component manages a **5-state authentication flow** using a single `authState` string variable:





6.2 Auth States — Detailed

State	UI Shown	Action	Next State
login	Email + Password fields, "Forgot Password?" link	POST <code>/api/login</code>	otp (if 2FA) or <code>/dashboard</code>
otp	6-digit OTP input, Resend Email (30s cooldown)	POST <code>/api/verify-otp</code>	<code>/dashboard</code>
forgot	Email field	POST <code>/api/forgot-password</code>	forgot-otp
forgot-otp	6-digit OTP input, Resend Email (30s cooldown)	POST <code>/api/verify-forgot-password-otp</code>	reset-password
reset-password	New Password + Confirm Password fields	POST <code>/api/update-password</code>	login

6.3 OTPInput Component

File: `features/auth/components/OTPInput.tsx`

A fully custom 6-digit OTP input component with the following features:

Feature	Description
Auto-focus	Focuses first input on mount
Auto-advance	Moves to next input after digit entry
Backspace support	Clears current digit or moves to previous input
Arrow key navigation	Left/Right arrow keys move between inputs
Paste support	Pasting a number string fills all inputs automatically
Auto-submit	Calls <code>onComplete</code> callback when all 6 digits are filled
Disabled state	Disables all inputs during loading

Props:

```
interface OTPInputProps {  
  length: number;           // Number of OTP digits (6)  
  value: string;             // Current OTP value  
  setValue: (value: string) => void;  
  onChange: (value: string) => void;  
  onComplete: (value: string) => void; // Called when all digits filled  
  disabled?: boolean;  
  containerClassName?: string;  
  inputClassName?: string;  
}
```

6.4 Resend OTP Cooldown Logic

Both login OTP and forgot-password OTP have a **30-second cooldown** timer before "Resend Email" becomes clickable again:

```
// Timer starts when authState changes to 'otp' or 'forgot-otp'  
useEffect(() => {  
  if (authState === "otp") {  
    setIsResendDisabled(true);  
    setResendTimer(30); // 30 second countdown  
  }  
}, [authState]);
```

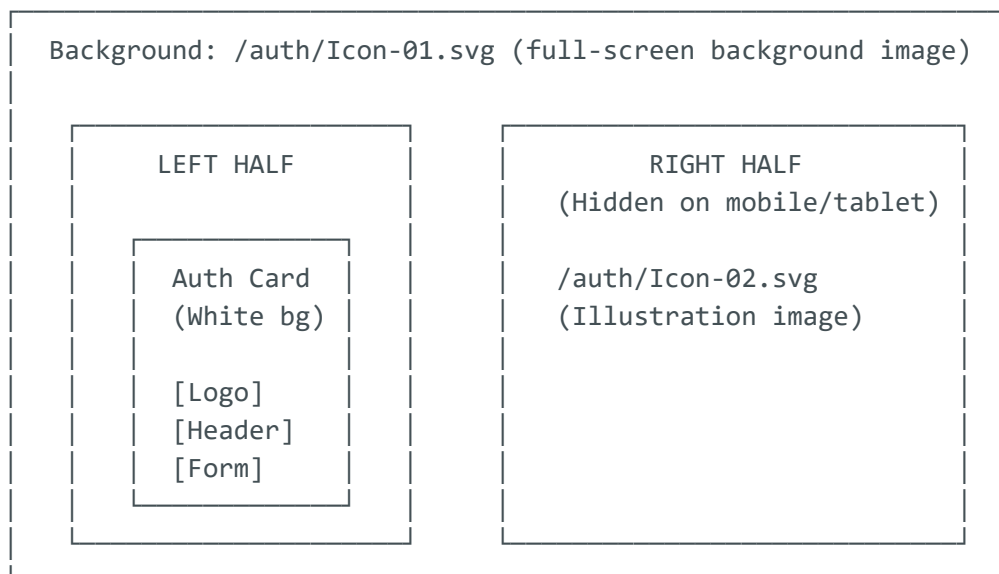
```

    }
  }, [authState]));

  // Countdown using useEffect + setTimeout
  useEffect(() => {
    if (resendTimer > 0) {
      timer = setTimeout(() => setResendTimer(resendTimer - 1), 1000);
    } else {
      setIsResendDisabled(false); // Re-enable button
    }
  }, [resendTimer]);

```

6.5 Auth Page Layout



7. Layout System

7.1 Root Layout (`app/layout.tsx`)

The root layout wraps the **entire application** and provides:

Provider	Purpose
Poppins font (Google Fonts)	Applied via CSS variable <code>--font-poppins</code>
<code><Toaster></code> (react-hot-toast)	Global toast notifications, positioned top-center

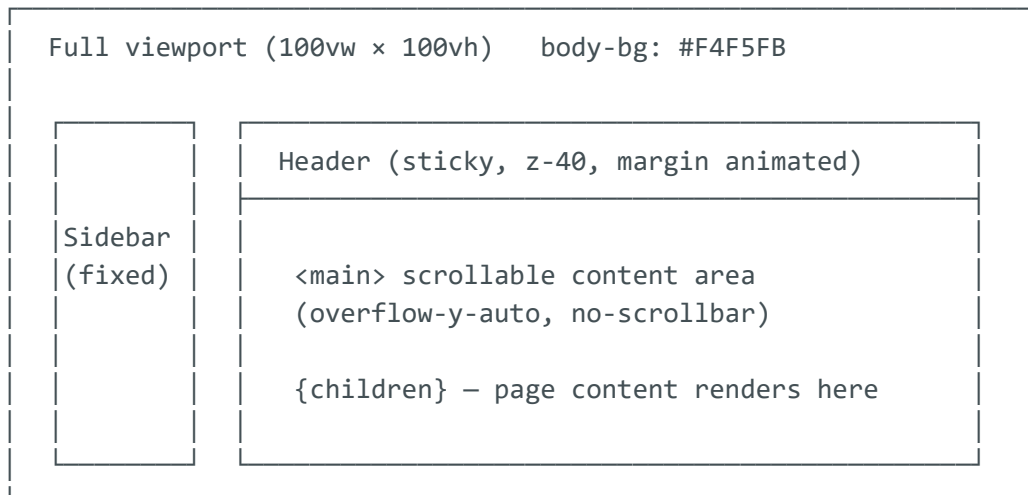
Provider	Purpose
<code><TooltipProvider></code> (Radix UI)	Required wrapper for all Tooltip components

Metadata:

```
export const metadata = {
  title: {
    template: '%s | SPC - PIL',    // e.g., "Dashboard | SPC - PIL"
    default: 'SPC - PIL',
  },
}
```

7.2 Authenticated Layout (`app/(authenticated)/layout.tsx`)

Applied to all protected pages. Structure:



Sidebar Toggle Animation (Framer Motion):

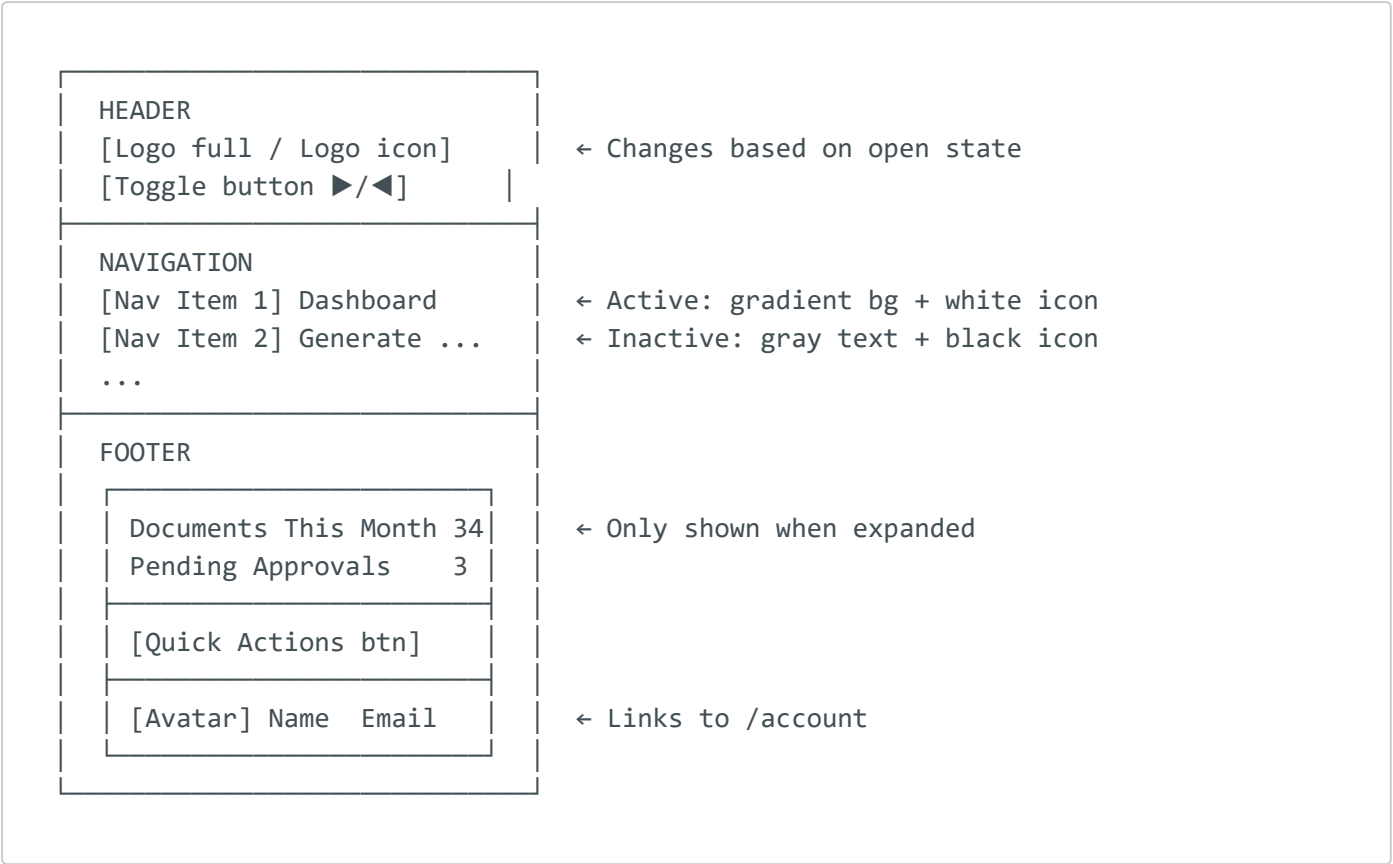
```
// Both Header and Main content animate their left margin
animate={{ marginLeft: open ? 255 : 75 }}
transition={{ duration: 0.6, ease: [0.4, 0, 0.2, 1] }}
```

7.3 Sidebar Component

(components/layout/Sidebar.tsx)

State	Width	Behavior
Collapsed	60px	Shows only icons; tooltips on hover show labels
Expanded	240px	Shows icons + text labels with fade-in animation

Sidebar Sections:



User Data Loading:

- On mount, reads `localStorage.getItem("user")`
- Parses JSON to extract `name`, `email`, `image`
- Displays avatar initials if no image (e.g., "Pranav Choudhary" → "PC")

Active Route Detection:

```
const isActive = (link: string) => {
  const linkPattern = new RegExp(`^${link}(\/|$)`);
  return linkPattern.test(pathname);
};
```

7.4 Header Component

(components/layout/Header.tsx)

The header **dynamically updates** its appearance based on the current route:

Route	Background	Icon	Text Color
/dashboard	White	Black icons	Black
All other routes	Gradient (teal→blue)	White icons	White

How it works:

```
// Matches current pathname to NavData to get title, icon, className, iconFolder
const { title, icon, className, iconFolder } = useMemo(() => {
  const matched = NavData.find((item) => {
    return new RegExp(`^${item.link}(/|$)`).test(pathname);
  });
  // Returns matched item's properties or fallback to "Dashboard"
}, [pathname]);
```

Header Contents:

- 1. **Page Icon** — SVG icon from /sidebar/{iconFolder}/{icon}
- 2. **Page Title** — Dynamic from NavData
- 3. **Search Input** — Styled to match header theme (white border on gradient, black on white)
- 4. **Action Buttons** — 2 icon buttons (filter/notification, profile — icons from /header/{iconFolder}/)

8. Dashboard Module

Location: features/dashboard/

8.1 Dashboard Page Composition

Dashboard Page

└ SearchDocument

(components/common/SearchDocument.tsx)

8.2 SearchDocument Component

File: `components/common/SearchDocument.tsx`

A gradient banner with document search functionality:

 Gradient Background (teal → blue)

"Search Document"
"Find the SPC or PIL document for your product"

 Search by product name..

Country

Authority

Type

Filter Options:

Filter	Options
Country	All Countries, USA, UK, Germany, France, Japan
Regulatory Authority	All Authorities, EMA, WHO, SFDA
Document Type	All Types, SPC, PIL

Each dropdown has an **inline search input** to filter the dropdown options in real-time.

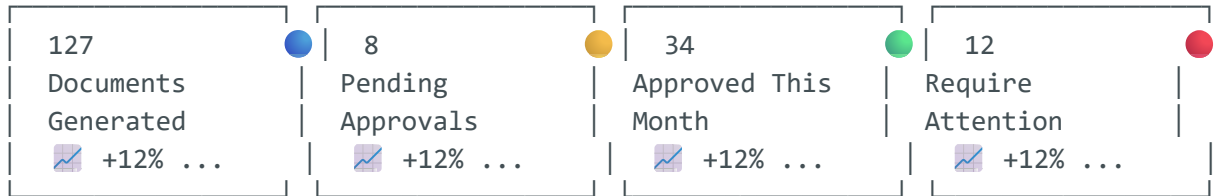
State managed:

- `searchQuery` — text input value
- `selectedCountry`, `selectedAuthority`, `selectedType` — dropdown selections
- `countrySearch`, `authoritySearch`, `typeSearch` — dropdown filter inputs

8.3 Cards Component

File: `features/dashboard/components/Cards.tsx`

Displays a 4-column grid of statistic cards:



CardData Interface:

```
interface CardData {  
  title: string;           // Card label  
  icon: string;            // SVG filename from /dashboard/cards/  
  iconBgColor: string;     // Hex color for icon background circle  
  value: string;           // Main stat number  
  stats: string;           // Trend description text  
  statsColor?: string;     // Color for trend indicator  
  isTrendingUp?: boolean;  // TrendingUp or TrendingDown icon  
}
```

9. API Layer

Location: `lib/apis/`

9.1 Architecture

The API layer currently uses a **Mock implementation** for development. The real backend API code is **commented out but ready** for production integration.

```
apicaller.ts  
├─ [COMMENTED] Real API Implementation (fetch-based, with JWT Bearer token)  
└─ [ACTIVE]    Mock API Implementation (in-memory, simulated delay)
```

9.2 `mainApiCaller` Function

Signature:

```
const mainApiCaller = async (
  endpoint: string,
  method: 'GET' | 'POST' | 'PUT' | 'DELETE' = 'GET',
  data?: any
): Promise<any>
```

Usage in components:

```
// Used with react-hot-toast's toast.promise() for automatic loading/success/error
toasts
toast.promise(
  mainApiCaller("/api/login", "POST", { email, password }),
  {
    loading: "Logging in...",
    success: (data) => { /* handle success */ return "Login Successful"; },
    error: (error) => error.message || "Something broke!",
  }
);
```

9.3 Mock API Endpoints

Endpoint	Method	Request Body	Response
/api/login	POST	{ email, password }	{ userId, twoFactorRequired } or { res: user }
/api/verify-otp	POST	{ userId, otp }	{ res: user, message }
/api/resend-otp	POST	{ userId }	{ message }
/api/forgot-password	POST	{ email }	{ userId, message }
/api/verify-forgot-password-otp	POST	{ userId, otp }	{ message }
/api/resend-forgot-password-otp	POST	{ userId }	{ message }

Endpoint	Method	Request Body	Response
/api/update-password	POST	{ userId, newPassword }	{ message }

9.4 Real API Implementation (Commented)

The real API code in `apicaller.ts` includes:

- Dynamic URL construction (prepends `/api` prefix if needed)
- `Content-Type: application/json` header
- **JWT Bearer Token** auth: reads from `localStorage.getItem('auth')`
- `credentials: 'include'` for cookie support
- Full error handling with HTTP status codes
- Response JSON parsing

To switch to real backend: **uncomment the real implementation** and **comment out the mock**.

9.5 Simulated API Delay

```
// All mock endpoints simulate 1 second network delay
export const simulateDelay = (ms: number = 1000): Promise<void> => {
  return new Promise(resolve => setTimeout(resolve, ms));
};
```

10. Component Library (UI)

Location: `components/ui/`

All UI components are based on **Shadcn/UI** (built on **Radix UI** primitives).

Button Component (`components/ui/button.tsx`)

Uses `class-variance-authority` for variant management.

Variants:

Variant	Appearance
<code>default</code>	Dark background, light text
<code>destructive</code>	Red background (for delete actions)
<code>outline</code>	Border only, hover changes bg
<code>secondary</code>	Light gray background
<code>ghost</code>	No background, hover shows bg
<code>link</code>	Text only with underline
<code>gradient</code>	Animated gradient (teal→blue, uses <code>button-gradient</code> class)
<code>gradient-outline</code>	Gradient border, becomes gradient bg on hover

Sizes: `xs`, `sm`, `default`, `lg`, `icon`, `icon-xs`, `icon-sm`, `icon-lg`

Card Component (`components/ui/card.tsx`)

Extended Shadcn Card with a custom `noborder` prop:

```
// noborder prop removes the default border (used in Auth form)
<Card noborder className="...">
```

Sub-components: `CardHeader`, `CardTitle`, `CardDescription`, `CardAction`, `CardContent`, `CardFooter`

Form Components

Uses **React Hook Form** + **Zod** integration via `@hookform/resolvers`:

```
const form = useForm({
  resolver: zodResolver(formSchema), // Zod schema validation
  defaultValues: { email: "", password: "", ... }
});
```

Form sub-components: `Form`, `FormControl`, `FormField`, `FormItem`, `FormLabel`, `FormMessage`

Utility Function (`lib/utils.ts`)

```
// Combines clsx (conditional classes) + tailwind-merge (dedup Tailwind classes)
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

11. Design System

11.1 Color Palette

Token	Value	Usage
<code>--border</code>	<code>#D4D3EA</code>	Default borders
<code>--background</code>	White	Page background
<code>.body-bg</code>	<code>#F4F5FB</code>	Authenticated page background
<code>.text-theme-secondary</code>	<code>#333333</code>	Secondary text
<code>.text-theme-sky</code>	<code>#17BDD3</code>	Sky blue accent (links, focus)
Gradient start	<code>#0469CE</code>	Blue
Gradient end	<code>#00CFAD</code>	Teal/Green

Dashboard Card Colors:

Card	Color	Hex
Documents Generated	Cyan	<code>#17BDD3</code>
Pending Approvals	Amber	<code>#FFB51D</code>
Approved This Month	Green	<code>#08DD7D</code>
Require Attention	Red	<code>#FF4949</code>

11.2 Gradient System

```
/* Background gradient (left to right: blue → teal) */
.bg-gradient {
  background: linear-gradient(269deg, #00cfad 0.74%, #0469ce 99.01%);
}

/* Hover-only gradient */
.bg-gradient-hover:hover {
  background: linear-gradient(269deg, #00cfad 0.74%, #0469ce 99.01%);
}

/* Button gradient with smooth rotation animation on hover */
.button-gradient {
  background: linear-gradient(90deg, #0469ce 0%, #00cfad 100%);
  animation: smoothRotateReverse 0.7s ease-in-out forwards; /* default state */
}
.button-gradient:hover {
  animation: smoothRotate 0.7s ease-in-out forwards; /* hover: rotates gradient 90°
→ 270° */
}
```

11.3 Typography

Font	Weights	Variable
Poppins (Google Fonts)	100–900	<code>--font-poppins</code>

Applied globally via:

```
body {
  font-family: var(--font-poppins), sans-serif;
}
```

11.4 Spacing & Radius

```
--radius: 0.425rem;
--radius-sm: calc(var(--radius) - 4px);
--radius-md: calc(var(--radius) - 2px);
--radius-lg: var(--radius);
--radius-xl: calc(var(--radius) + 4px);
```

11.5 Dark Mode

Dark mode is supported via CSS class strategy (**dark** class on parent):

```
@custom-variant dark (&:is(.dark *));
```

All tokens have dark mode overrides defined in the **.dark {}** block in **globals.css**.

11.6 Custom Scrollbar

```
.custom-scrollbar /* Applied on body – custom styled scrollbar */  
.no-scrollbar    /* Applied on sidebar nav + main content – hides scrollbar  
visually */
```

12. Navigation Configuration

File: **lib/NavData.ts**

All navigation items are centrally defined here and consumed by both **Sidebar** and **Header** components.

```
export interface NavItem {  
  title: string;      // Display name  
  icon: string;       // SVG filename (e.g., "Icon-01.svg")  
  link: string;       // Route path  
  hidden: boolean;    // If true, hidden from sidebar nav  
  className: string;  // Header background class for this route  
  iconFolder?: string; // "black" or "white" – which icon set to use in header  
}
```

Full Navigation Items:

#	Title	Route	Icon	Hidden	Header Theme
1	Dashboard	<code>/dashboard</code>	Icon-01	✗	White bg, black icons
2	Generate SPC / PIL	<code>/generate-SPC-PIL</code>	Icon-02	✗	Gradient bg, white icons
3	Document Editor	<code>/document-editor</code>	Icon-03	✗	Gradient bg, white icons
4	Workflow Management	<code>/workflow-management</code>	Icon-04	✗	Gradient bg, white icons
5	Document Repository	<code>/document-repository</code>	Icon-05	✗	Gradient bg, white icons
6	Template Library	<code>/template-library</code>	Icon-06	✗	Gradient bg, white icons
7	Regulatory Intelligence	<code>/regulatory-intelligence</code>	Icon-07	✗	Gradient bg, white icons
8	Compare Documents	<code>/compare-documents</code>	Icon-08	✗	Gradient bg, white icons
9	Audit & Version History	<code>/audit-version-history</code>	Icon-09	✗	Gradient bg, white icons
10	Notification Center	<code>/notification-center</code>	Icon-10	✗	Gradient bg, white icons
11	Settings	<code>/settings</code>	Icon-11	✗	Gradient bg, white icons
12	Account	<code>/account</code>	Icon-13	✓	Gradient bg, white icons

Note: Account is `hidden: true` — it doesn't appear in the sidebar nav list, but the route exists and is linked from the user avatar in the sidebar footer.

13. Mock Data & Dev Credentials

Test User Accounts

Email	Password	Name	2FA Required	Role
test.u@ddreg.in	test123	Pranav Choudhary	☒ Yes	user
admin@example.com	admin123	Admin User	☒ No	admin
demo@example.com	demo123	Demo User	☒ Yes	user

OTP (All Users)

Development OTP: 123456 (hardcoded in generateOTP())

The actual OTP is also logged in the browser console: Generated OTP for user {id}: 123456

Dashboard Mock Data

Card	Value	Color
Documents Generated	127	Cyan #17BDD3
Pending Approvals	8	Amber #FFB51D
Approved This Month	34	Green #08DD7D
Require Attention	12	Red #FF4949

14. Environment & Configuration

TypeScript Config (tsconfig.json)

Option	Value	Purpose
target	ES2017	Compile to ES2017

Option	Value	Purpose
<code>strict</code>	<code>true</code>	Strict type checking
<code>paths</code>	<code>"@/*": ["..//*"]</code>	Absolute imports using <code>@/</code> prefix
<code>moduleResolution</code>	<code>bundler</code>	Next.js bundler resolution

Path Alias

All imports use the `@/` alias which resolves to the project root:

```
// Instead of relative imports like:
import { Button } from "../../components/ui/button"

// Use absolute alias:
import { Button } from "@components/ui/button"
```

Shadcn Config (`components.json`)

Manages Shadcn/UI component generation settings (component paths, style, aliases).

Next.js Config (`next.config.ts`)

Currently minimal — no custom configuration added yet.

15. Scripts & Commands

```
# Start development server (hot reload)
npm run dev

# Build for production
npm run build

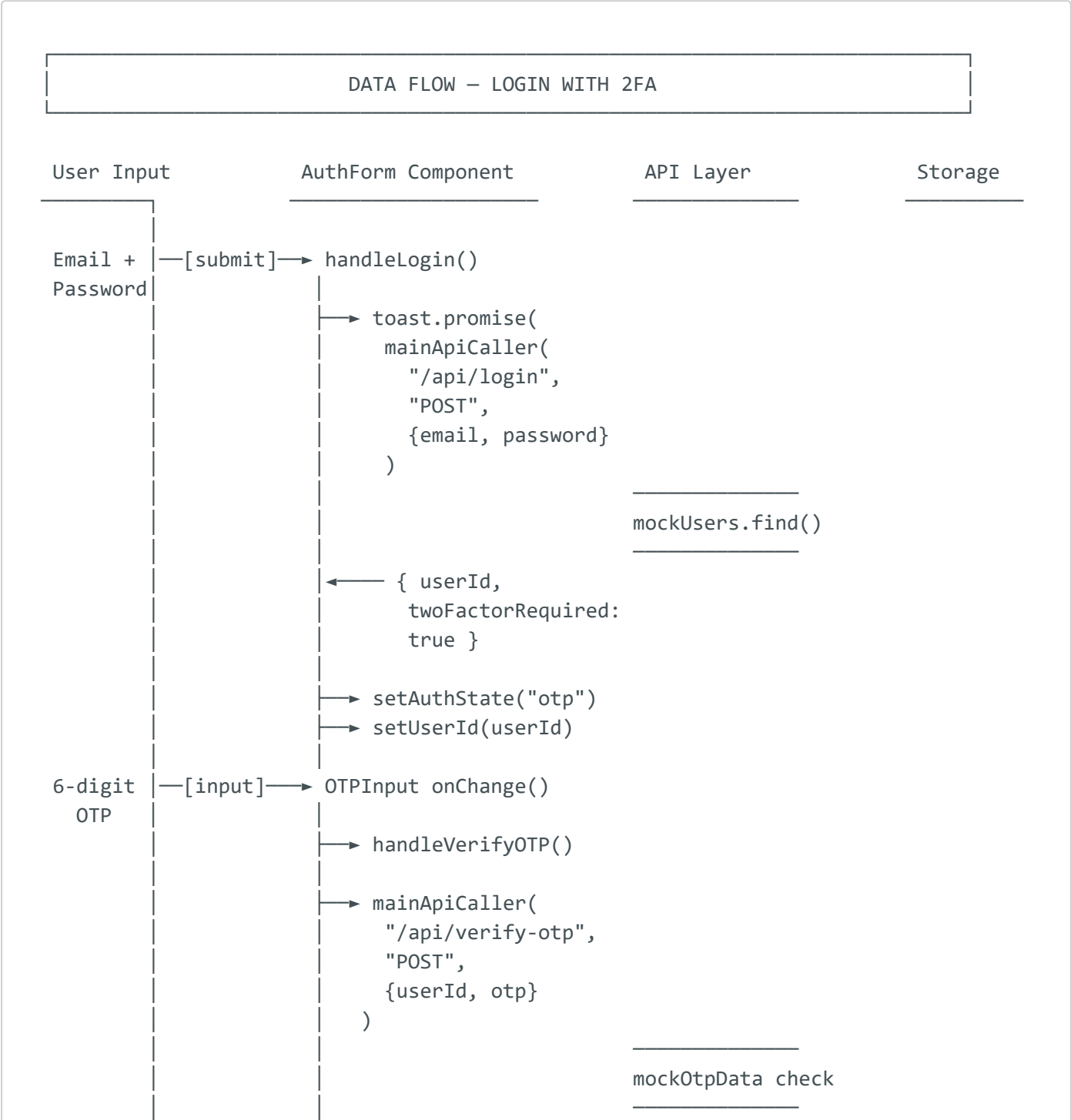
# Start production server (requires build first)
npm run start
```

```
# Run ESLint
npm run lint
```

Development Server

- Runs on: <http://localhost:3000>
- Default route (/) shows Login page

Appendix: Data Flow Diagram





📌 **Documentation Last Updated:** February 2026

📁 **Repository:** [spc-pil](#) (DDReg Pharma Internal)

💻 **Stack:** Next.js 16 + TypeScript + Tailwind CSS v4 + Shadcn/UI